

Group 2 — API Integration Guide

Government Asset Management System

From: Group 1 (Administration, Security & Multi-tenancy) **To:** Group 2 (Asset & Property Register Module)

Purpose: Single Sign-On authentication, user identity, role-based access, and multi-tenant data isolation

Table of Contents

1. [What This Document Covers](#)
 2. [What We Need From Group 2 Before Integration Works](#)
 3. [Architecture Overview — How Login Works](#)
 4. [Keycloak Configuration — Endpoints Group 2 Needs](#)
 5. [The Complete Login Flow \(Step by Step\)](#)
 6. [API Endpoint: Verify Token — Get User Role and Ministry](#)
 7. [API Endpoint: Logout — End Session](#)
 8. [Understanding the JWT Token \(What's Inside\)](#)
 9. [Role Definitions — The 5 Permission Levels](#)
 10. [Test Accounts](#)
 11. [Ministries Available](#)
 12. [Error Scenarios — What to Expect and How to Handle](#)
 13. [Security Rules Group 2 Must Follow](#)
 14. [Integration Checklist](#)
-

1. What This Document Covers

This document tells Group 2 everything they need to connect their system to ours. By the end, Group 2 will be able to:

- Let their users log in using our **government SSO** (Keycloak login page)
- Know the user's **identity** (name, username, email)
- Know the user's **role** (what they are allowed to do)
- Know the user's **ministry** (which ministry's data to show)
- Log the user **out** from all systems at once

What Group 2 does NOT need from us:

- Asset CRUD (create, read, update, delete) — you have your own
- Asset categories — you have your own
- Org hierarchy — you have your own
- Audit logs — you log your own actions
- Dashboard stats — you build your own

We only provide what you cannot build yourself: authentication, identity, role, and ministry context.

2. What We Need From Group 2 Before Integration Works

Before anything works, Group 2 must send us these **3 things** so we can register their application in our Keycloak server:

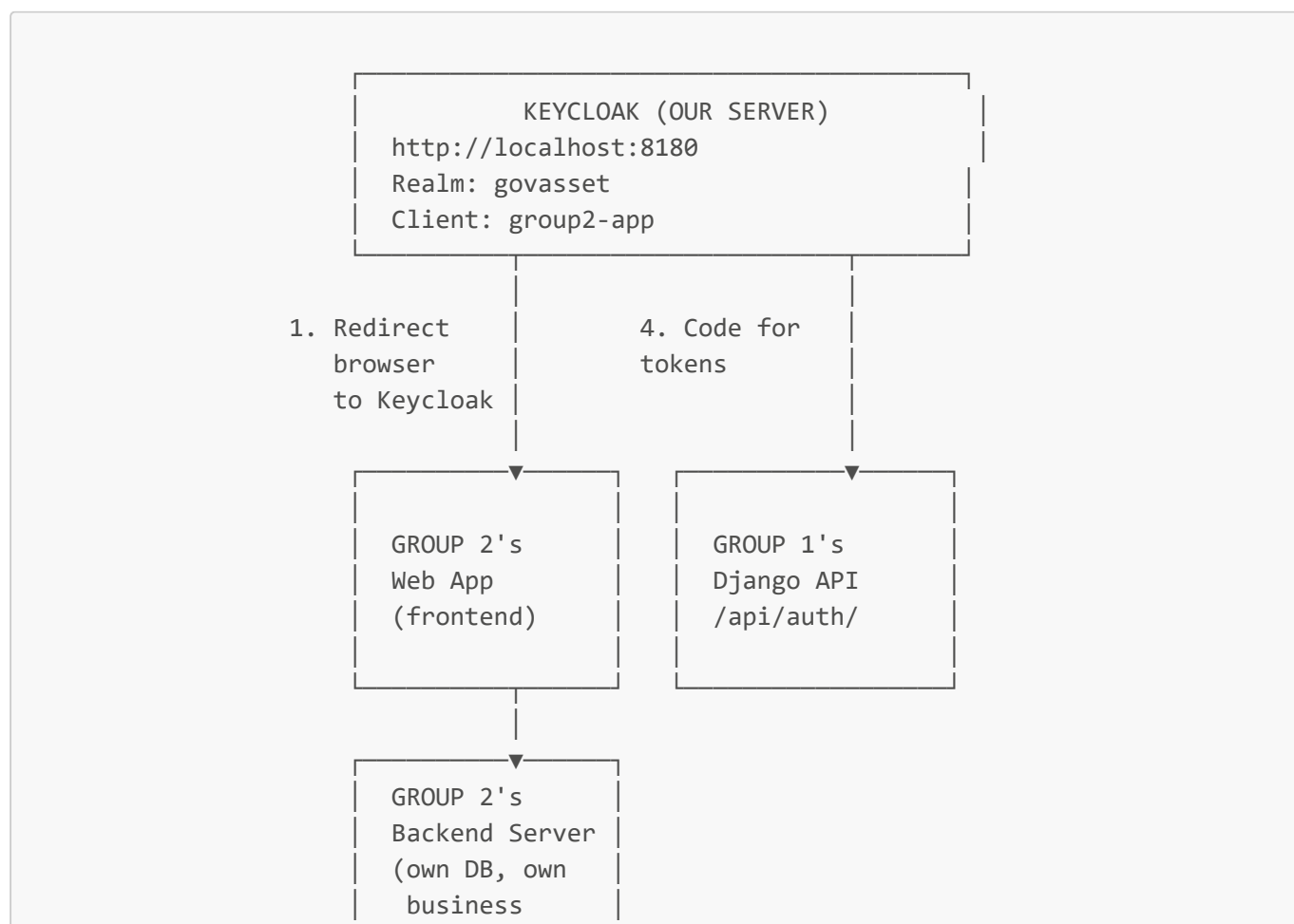
#	What we need	Example	Why
1	Application Name	Group 2 Asset Register	To label their client in our Keycloak admin panel
2	Login Redirect URI	https://group2-system.com/auth/callback	Where Keycloak sends the user after successful login
3	Logout Redirect URI	https://group2-system.com	Where Keycloak sends the user after logout

Once we receive these, we will:

1. Create an OIDC client in our Keycloak for Group 2
2. Generate a **Client ID** and **Client Secret**
3. Share the complete configuration with Group 2

3. Architecture Overview — How Login Works

Group 2's system does NOT have its own login page. Every user must log in through our centralized Keycloak SSO.



```
logic)
```

What Group 2 needs to implement on their side:

Component	What to build
Login button	Redirects user's browser to our Keycloak auth endpoint
Callback handler	Receives the auth code from Keycloak, exchanges it for tokens
Token storage	Keeps the access token in memory or secure session
API call to our verify endpoint	Calls <code>GET /api/auth/verify-token/</code> to get role + ministry
Logout button	Redirects user's browser to Keycloak logout endpoint

4. Keycloak Configuration — Endpoints Group 2 Needs

These are the Keycloak endpoints Group 2 will interact with. They will not change.

What	URL
Keycloak Server	<code>http://localhost:8180</code>
Realm	<code>govasset</code>
Authorization Endpoint	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/auth</code>
Token Endpoint	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/token</code>
JWKS URI (public keys)	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/certs</code>
Userinfo Endpoint	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/userinfo</code>
Logout Endpoint	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/logout</code>

Values we will generate and share (after Group 2 sends us their redirect URIs):

Setting	Value (example)
Client ID	<code>group2-app</code>
Client Secret	<code>(will be generated by Keycloak)</code>

5. The Complete Login Flow (Step by Step)

Step 1 — User clicks "Login" on Group 2's website

Group 2's frontend redirects the user's browser to:

```
GET http://localhost:8180/realms/govasset/protocol/openid-connect/auth?
  client_id=group2-app&
  redirect_uri=https://group2-system.com/auth/callback&
  response_type=code&
  scope=openid+profile+email&
  state=RANDOM_STRING
```

Query parameter explanation:

Parameter	Value	Why
<code>client_id</code>	<code>group2-app</code>	Identifies Group 2's application to Keycloak
<code>redirect_uri</code>	<code>https://group2-system.com/auth/callback</code>	Where to send the user after login (must match what you gave us)
<code>response_type</code>	<code>code</code>	Requests the authorization code flow
<code>scope</code>	<code>openid profile email</code>	Asks for user identity info
<code>state</code>	Random string	Prevents CSRF attacks — verify it matches when the user returns

Step 2 — User sees our Keycloak login page

The user sees the government-branded login page in their browser. They type their username and password.

Step 3 — Keycloak redirects back to Group 2

After successful login, Keycloak redirects the browser to:

```
https://group2-system.com/auth/callback?code=abc123&state=RANDOM_STRING
```

Important: Group 2 must verify that the `state` value matches what they sent in Step 1.

Step 4 — Group 2's backend exchanges the code for tokens

Group 2's backend makes a server-to-server call (NOT from the browser):

```
POST http://localhost:8180/realms/govasset/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&client_id=group2-app
&client_secret=THE_CLIENT_SECRET
```

```
&code=abc123
&redirect_uri=https://group2-system.com/auth/callback
```

Step 5 — Keycloak returns tokens

Success Response (HTTP 200):

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "refresh_token": "eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "not-before-policy": 0,
  "session_state": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "scope": "openid profile email"
}
```

What each token is for:

Token	Purpose	Expiry
access_token	Call our API. Contains user claims (role, ministry_schema) embedded in the JWT.	5 minutes
refresh_token	Get a new access token when it expires.	30 minutes
id_token	Contains user profile info (name, email). Not used for API calls.	5 minutes

Step 6 — Group 2 calls our API to get user role and ministry

(See Section 6 below for full details)

Step 7 — Group 2 shows their dashboard

Group 2 now knows:

- Who the user is (full_name, username)
- What they can do (role)
- Which ministry's data to show (ministry_schema, ministry)

6. API Endpoint: Verify Token — Get User Role and Ministry

This is the **only Group 1 API endpoint** Group 2 needs to call. It validates the user's access token and returns their complete profile including role and ministry.

Request

```
GET http://localhost:8000/api/auth/verify-token/
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

Headers:

Header	Value	Required?
Authorization	Bearer <access_token>	Yes

Success Response (HTTP 200)

```
{
  "valid": true,
  "user": {
    "id": 29,
    "username": "moh_admin",
    "full_name": "Amina Hassan",
    "email": "amina@moh.go.tz",
    "role": "MINISTRY_ADMIN",
    "ministry_schema": "moh_schema",
    "ministry": "Ministry of Health"
  }
}
```

Field explanations:

Field	Example	What it means
user.id	29	Internal user ID in our database
user.username	moh_admin	Login username (unique across entire system)
user.full_name	Amina Hassan	User's real name — display this in your UI header
user.email	amina@moh.go.tz	User's email address
user.role	MINISTRY_ADMIN	Permission level — use this to control what the user can see/do in your UI (see Section 9 for all 5 roles)
user.ministry_schema	moh_schema	Internal schema name — use this to scope database queries if you read from our database
user.ministry	Ministry of Health	Display name — show this in your UI to indicate which ministry the user belongs to

Error Response (HTTP 401)

```
{
  "detail": "Given token not valid for any token type"
```

```
}
```

The token has expired or is invalid. Group 2 should:

1. Try to refresh the token using the refresh token (see Step 5.1 below)
2. If refresh fails, redirect the user back to the Keycloak login page

Step 5.1 — Token Refresh (When Access Token Expires)

If the verify-token call returns 401, Group 2 can get a new access token using the refresh token:

```
POST http://localhost:8180/realms/govasset/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&client_id=group2-app
&client_secret=THE_CLIENT_SECRET
&refresh_token=eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9...
```

Success Response (HTTP 200):

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs...",
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "refresh_token": "eyJhbGciOiJIUzUxMiIs...",
  "token_type": "Bearer"
}
```

Error Response (HTTP 400): The refresh token has also expired. Group 2 must redirect the user back to the Keycloak login page.

7. API Endpoint: Logout — End Session

When the user clicks "Logout" on Group 2's system, redirect their browser to:

```
GET http://localhost:8180/realms/govasset/protocol/openid-connect/logout?
id_token_hint=eyJhbGciOiJSUzI1NiIs...&
post_logout_redirect_uri=https://group2-system.com
```

Query parameter explanation:

Parameter	Value	Why
<code>id_token_hint</code>	The ID token from Step 5	Tells Keycloak which session to end

Parameter	Value	Why
<code>post_logout_redirect_uri</code>	<code>https://group2-system.com</code>	Where to redirect after logout

After logout, Keycloak redirects the browser to `https://group2-system.com`. **This logs the user out of all connected systems at once** (single sign-out).

8. Understanding the JWT Token (What's Inside)

The access token that Keycloak returns is a **JWT (JSON Web Token)**. Group 2 can decode it to read the embedded user information without calling any API.

Decoded access token payload:

```
{
  "exp": 1712345678,
  "iat": 1712345378,
  "auth_time": 1712345378,
  "jti": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "iss": "http://localhost:8180/realms/govasset",
  "aud": "account",
  "sub": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "typ": "Bearer",
  "azp": "group2-app",
  "session_state": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "acr": "1",
  "realm_access": {
    "roles": ["default-roles-govasset", "offline_access", "uma_authorization"]
  },
  "resource_access": {
    "account": {
      "roles": ["manage-account", "manage-account-links", "view-profile"]
    }
  },
  "scope": "openid profile email",
  "sid": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "email_verified": true,
  "role": "MINISTRY_ADMIN",
  "ministry_schema": "moh_schema",
  "preferred_username": "moh_admin",
  "given_name": "Amina",
  "family_name": "Hassan",
  "email": "amina@moh.go.tz"
}
```

The two most important custom claims:

- `"role": "MINISTRY_ADMIN"` — embedded by our Keycloak user attributes
- `"ministry_schema": "moh_schema"` — embedded by our Keycloak user attributes

Group 2 can either:

- 1. **Decode the JWT themselves** — read `role` and `ministry_schema` directly from the token (no API call needed, but may be stale if admin changes role)
- 2. **Call our verify-token API** — always returns the latest data (recommended for critical permission checks)

9. Role Definitions — The 5 Permission Levels

Our system has exactly **5 roles**. Every user in the entire 10-group project has one of these roles. Group 2 should use the `role` field to control what each user can do in their UI.

Role	Level	Who has this role	What they can do (across the whole system)
<code>SUPER_ADMIN</code>	5 (highest)	System Administrators	Full access to everything — all ministries, all data, all settings. Only 1-2 people have this role.
<code>MINISTRY_ADMIN</code>	4	Ministry IT Administrators	Full access within their own ministry. Can manage users, create/edit any asset, view audit logs.
<code>AGENCY_MANAGER</code>	3	Agency Heads (e.g., Hospital Director)	Can manage assets within their agency, create Facility Clerk accounts, view reports.
<code>FACILITY_CLERK</code>	2	Frontline staff (e.g., Store Clerk)	Can register new assets, view assets, edit assets they created. Cannot delete or manage users.
<code>AUDITOR</code>	1 (lowest)	Government Auditors	Read-only access. Can view all assets and audit logs. Cannot create, edit, or delete anything.

How Group 2 should use roles in their UI:

UI Element	Show to
"Create Asset" button	MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK
"Edit Asset" button	MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK (own assets)
"Delete Asset" button	MINISTRY_ADMIN only
"View Audit Log" button	MINISTRY_ADMIN, AUDITOR
"Manage Users" button	MINISTRY_ADMIN, AGENCY_MANAGER (limited)
"Dashboard with all ministries"	SUPER_ADMIN only
"Export Report" button	All roles (but scope to their ministry)

10. Test Accounts

All 6 accounts use the same password: `Admin@123`

Username	Full Name	Role	Ministry	Keycloak Available?
<code>superadmin</code>	System Administrator	SUPER_ADMIN	All Ministries	Yes
<code>moh_admin</code>	Amina Hassan	MINISTRY_ADMIN	Ministry of Health	Yes
<code>mnh_manager</code>	John Mwangi	AGENCY_MANAGER	Ministry of Health	Yes
<code>rad_clerk</code>	Asha Salum	FACILITY_CLERK	Ministry of Health	Yes
<code>moh_auditor</code>	David Mushi	AUDITOR	Ministry of Health	Yes
<code>mof_admin</code>	Grace Mbwilo	MINISTRY_ADMIN	Ministry of Finance	Yes

Group 2 can use any of these accounts to test their integration.

- Log in as `moh_admin` → should see "Ministry of Health" scoped data
- Log in as `mof_admin` → should see "Ministry of Finance" scoped data
- Log in as `superadmin` → can see all ministries

11. Ministries Available

Ministry	Schema Name	Users Currently
Ministry of Health	<code>moh_schema</code>	<code>moh_admin</code> , <code>mnh_manager</code> , <code>rad_clerk</code> , <code>moh_auditor</code>
Ministry of Finance	<code>mof_schema</code>	<code>mof_admin</code>
Super Admin (platform-wide)	None (public schema)	<code>superadmin</code>

Multi-tenancy rule (important): When a user with `ministry_schema: "moh_schema"` is logged in, Group 2 must:

- Show ONLY Ministry of Health data
- Hide Ministry of Finance data
- Hide Ministry of Finance users from user lists
- Apply this filtering in Group 2's own database queries

12. Error Scenarios — What to Expect and How to Handle

Scenario 1: User types wrong password on Keycloak page

Keycloak shows an error message on the login page: "Invalid username or password." After 5 failed attempts, the account is locked.

Group 2 does nothing — this is handled entirely by Keycloak. Group 2's callback will not be called.

Scenario 2: Auth code expires before exchange

Auth codes are valid for **60 seconds**. If Group 2's backend takes too long to call the token endpoint:

```
POST http://localhost:8180/realms/govasset/protocol/openid-connect/token
```

Response (HTTP 400):

```
{
  "error": "invalid_grant",
  "error_description": "Code is expired"
}
```

Fix: Redirect the user back to the Keycloak login page again. The user will not need to re-enter their password if their Keycloak session is still active (they will be immediately redirected back with a new code).

Scenario 3: Access token expired

Group 2 calls our API:

```
GET /api/auth/verify-token/
Authorization: Bearer <expired_token>
```

Response (HTTP 401):

```
{
  "detail": "Given token not valid for any token type"
}
```

Fix: Use the refresh token at the Keycloak token endpoint (see Section 5.1).

Scenario 4: User's account is deactivated by an admin

The user logs in successfully on Keycloak (Keycloak doesn't know about deactivation). Group 2 calls our verify-token endpoint.

Response (HTTP 401):

```
{
  "detail": "Given token not valid for any token type"
}
```

Fix: Group 2 should treat this like an expired token and redirect to the Keycloak login page. Keycloak will show the login page, but the user's session may still be active. If the user keeps getting redirected in a loop, their account has likely been deactivated — they should contact their Ministry Administrator.

Scenario 5: Our server is down

Group 2 calls our verify-token endpoint but gets a network error (connection refused, timeout).

Fix: Group 2 should:

1. Show a friendly error message: *"Authentication service is temporarily unavailable. Please try again later."*
2. Keep the user's current session alive (don't log them out)
3. Retry after a few seconds
4. Optionally, decode the JWT locally (from the access token) as a fallback — the token's `role` and `ministry_schema` claims can still be trusted within the token's expiry window

Scenario 6: Group 2 receives a token for a user that doesn't exist in our database

This should not happen if users are created through our system. But if it does, verify-token will return:

Response (HTTP 401):

```
{
  "detail": "User not found"
}
```

Fix: The user exists in Keycloak but not in our Django database. Contact Group 1 to create the user properly.

13. Security Rules Group 2 Must Follow

1. **Never store user passwords.** Group 2 never handles passwords — that is Keycloak's job. If Group 2's login form sends passwords to their own backend, that is a security violation.
2. **Never share the Client Secret.** The `client_secret` is like a password for Group 2's application. It must never be exposed in frontend code, browser JavaScript, or mobile app code. Only Group 2's backend server should know it.
3. **Always validate the `state` parameter.** When Keycloak redirects back to Group 2's callback, verify that the `state` value matches what was sent in the original auth request. This prevents CSRF attacks.
4. **Use HTTPS in production.** The OIDC flow sends tokens in URLs and HTTP headers. In production, all communication must be over HTTPS. During development, HTTP to localhost is acceptable.

5. **Validate tokens on your backend.** Token verification should happen on Group 2's backend server, not in browser JavaScript. Never trust a token that was not verified by Group 2's server.
 6. **Respect the `role` field.** If a user with `role: "FACILITY_CLERK"` tries to perform an action that requires `MINISTRY_ADMIN`, Group 2 must reject the request. Do not rely on hiding buttons alone — enforce permissions on the backend.
 7. **Scope data by `ministry_schema`.** If a request comes in for Ministry of Finance data from a user whose `ministry_schema` is `moh_schema`, reject it. Data isolation between ministries is mandatory.
-

14. Integration Checklist

Before starting — Group 2 sends us:

- ☐ Application name
- ☐ Login redirect URI
- ☐ Logout redirect URI

We send back to Group 2:

- ☐ Client ID
- ☐ Client Secret
- ☐ This document ✓ (already have it)

Group 2 implements:

- ☐ Login button redirects to Keycloak auth endpoint
 - ☐ Callback handler exchanges code for tokens
 - ☐ `GET /api/auth/verify-token/` called after every login
 - ☐ `user.role` used to control permissions
 - ☐ `user.ministry` used to scope data
 - ☐ Token refresh implemented (5-min access token expiry)
 - ☐ Logout button redirects to Keycloak logout
 - ☐ Error handling for expired tokens, server down, etc.
 - ☐ Backend-enforced permission checks (not just UI hiding)
 - ☐ Data scoped by `ministry_schema`
-

End of document. For questions, contact Group 1 (Administration, Security & Multi-tenancy).